PRODUCTION MASTERPLAN

Prometheus & Grafana: YouTube Creator Blueprint

A Complete Production Roadmap, Screencast Architecture, PromQL Pedagogical Framework, and Channel Strategy

COURSE DURATION	TARGET AUDIENCE	TARGET RETENTION	LAB ARCHITECTURE
120–150 Mins	**SREs, DevOps & Devs**	**42%+ AVD**	**Docker Compose**
Single Mega-Crash Course or 5-Part Series	Familiar with Docker & REST APIs	Live load spike dashboards hold attention	Prometheus + Node Exporter + App + Grafana

4-Week Video Production & Delivery Timeline

EXECUTION WORKFLOW

Phase	Focus Area	Key Actions & Deliverables	Milestone
Week 1	Sandbox & App Instrumentation	Build a lightweight FastAPI/Node.js app with simulated load spikes, error triggers, and client library metrics. Assemble reproducible `compose.yaml`.	Demo Sandbox Ready
Week 2	Visual Decks & PromQL Cheatsheet	Create animated vector diagrams illustrating: Pull vs Push architecture, TSDB chunk storage, Counter resets, and Rate calculation mechanics.	Visual Decks Locked
Week 3	Studio Screencast Capture	Record in 1440p/4K 60fps. Capture side-by-side terminal load generators (`hey` or `k6`) paired with real-time Grafana dashboard refreshes (1s interval).	Raw Footage Complete
Week 4	Post-Production & Launch	Add zoom cuts onto PromQL vector selectors and Alertmanager routing rules; design 3 high-CTR thumbnails; publish tagged companion GitHub repo.	Published & Promoted

Recording Environment, UI Scaling & Retention Engineering

PRODUCTION QUALITY

Grafana & Terminal Screen Standard

- **Grafana Display:** Set Grafana UI to Dark Mode with 110%–125% zoom for high readability on phones.
- **Auto-Refresh Rate:** Set dashboard auto-refresh to 5s (or 1s during live traffic simulations).
- **CLI Fonts:** JetBrains Mono at 18pt–20pt; clean two-line Starship terminal prompt without excessive telemetry.
- **Multi-Pane Layout:** Left: Grafana Panel Builder; Right: Code / Terminal executing load generation scripts.

YouTube Retention Engineering

- **First 45 Seconds:** Trigger a traffic blast that knocks down an app; watch Grafana turn red and alert fire in real-time.
- **Demystify rate():** Dedicate an animated segment to `rate()` and `irate()` —the #1 stumbling block in PromQL.
- **Golden Signals Anchor:** Frame the entire tutorial around Google SRE Golden Signals (Latency, Traffic, Errors, Saturation).
- **Downloadable Asset:** Promote a 1-page PromQL + Grafana cheat sheet PDF in the pinned comment.

Pedagogical Core Principle: "Cause and Effect Observability"

Never show static zero-traffic dashboards. Every metric introduced must be coupled with an immediate real-world action (e.g., throwing 500 errors or maxing CPU cores with `stress-ng`) so viewers physically see the query respond to cluster state changes.

Curriculum Breakdown: Modules 1 to 3

METRICS ARCHITECTURE TO PROMQL

Module 1: The Observability Landscape & Prometheus Architecture

Timing: 25 Mins

- **The 3 Pillars of Observability:** Metrics, Logs, and Traces (when to use metrics vs centralized logs like Loki).
- **Pull vs Push Paradigm:** Why Prometheus scrapes targets via HTTP (`/metrics`) instead of passive agent ingestion; service discovery mechanics.
- **Prometheus Core Components:** Server (Retrieval Engine + TSDB), Scrape Targets, Exporters, Pushgateway (batch jobs), and Alertmanager.
- **Metric Data Types:** Deep dive with visual analogies: `Counter` (always increasing), `Gauge` (fluid value), `Histogram` (buckets), and `Summary` .
- **Docker Compose Lab Deployment:** Orchestrating Prometheus, Node Exporter, and cAdvisor in one command.

Module 2: Exporters & Application Instrumentation

Timing: 30 Mins

- **Blackbox vs Whitebox Monitoring:** Probing public endpoints vs internal application telemetry.
- **System Metrics with Node Exporter:** CPU utilization, memory pressure, disk I/O wait, and network throughput metrics.
- **Container Metrics with cAdvisor:** Per-container CPU throttling, memory RSS, and network socket activity.
- **Instrumenting Custom Code:** Adding client libraries (`prom-client` or `prometheus_client` Python) to expose HTTP request duration and error counters.
- **Configuring prometheus.yml:** Scrape intervals, target configurations, static targets vs dynamic discovery, and relabeling rules.

Module 3: PromQL Mastery (Prometheus Query Language)

Timing: 35 Mins

- **Selectors & Matchers:** Instant vectors vs Range vectors (`[5m]`); label equality, regex filtering (`=~` , `!~`).
- **Mastering rate() & irate():** How `rate()` calculates per-second average rate of increase; handling Counter resets after pod restarts.
- **Aggregation Operators:** Grouping metrics by dimension: `sum() by (status_code)` , `avg() without (instance)` , `topk()` , and `count()` .
- **Mathematical Vector Matching:** Combining multi-dimensional vectors using `on()` , `ignoring()` , `group_left` , and `group_right` .
- **Calculating Error Percentages:** Writing the canonical production SRE query: `(sum(rate(http_requests_total{status=~"5.."}[5m])) / sum(rate(http_requests_total[5m]))) * 100` .

Creator Tip: Live PromQL Sandbox Visualizer

Use the native Prometheus expression browser (`http://localhost:9090/graph`) in table view before charting in Grafana. Show how range vectors `[1m]` contain arrays of timestamps, and how `rate()` flattens them into a clean instant scalar float.

Curriculum Breakdown: Modules 4 to 6

DASHBOARDS, ALERTING & PRODUCTION

Module 4: Professional Grafana Dashboards & Visualizations

Timing: 25 Mins

- **Data Source Integration:** Securely connecting Prometheus datasource, timeout configurations, and query caching.
- **Panel Anatomy:** Choosing the right visualization: Time Series charts, Stat gauges, Heatmaps, Bar Gauges, and Logs panels.
- **Dynamic Dashboard Templating:** Creating interactive multi-tenant dropdown variables (`$environment` , `$node` , `$service`).
- **Importing Production Dashboards:** Discovering, customizing, and repairing community dashboards (e.g., Node Exporter Full ID `1860`).

Module 5: Alertmanager, Thresholds & Incident Routing

Timing: 20 Mins

- **Defining Prometheus Alerting Rules:** Writing declarative YAML alert files with `expr` , `for: 2m` duration, severity labels, and annotations.
- **Alertmanager Architecture:** Grouping alerts, deduplication, silences, and inhibit rules (suppressing app alerts if host node is dead).
- **Notification Receivers:** Routing live webhook payloads to Discord, Slack, PagerDuty, and email.
- **Grafana Managed Alerts vs Prometheus Native:** Comparing alerting in the Grafana UI vs immutable GitOps alert rule files.

YouTube Packaging, Titles, SEO & Promotion Strategy

GROWTH & DISTRIBUTION

Titles, Packaging & SEO

• **Title Option 1 (High CTR):** "Prometheus & Grafana Full Course: Monitor Anything Like a Pro"

• **Title Option 2 (Action-Driven):** "Never Let Production Crash Unnoticed: Observability Masterclass"

• **Target Search Keywords:** prometheus tutorial, grafana dashboard setup, learn promql, devops monitoring, alertmanager slack.

Thumbnail Strategy & Growth Assets

• **Thumbnail Layout:** Prometheus flame + Grafana diamond + dark dashboard with glowing red spikes + text: "ZERO BLIND SPOTS".

• **Pinned Comment Asset:** Link to ready-to-run GitHub repo with `docker compose up` setup + 1-page PromQL cheat sheet.

• **Shorts/Clips Repurposing:** Cut 3 clips: "The difference between rate() and irate()", "Why Prometheus pulls instead of pushes", "How to alert Slack in 60s".

Creator Deliverable: The "1-Click Cluster Monitoring" Repo

Ensure your companion repository has an all-in-one `compose.yaml` file with Prometheus, Grafana, Node Exporter, Alertmanager, and pre-provisioned datasource/dashboard YAML files. Viewers can clone and run it within 10 seconds without any manual GUI setup.